Fundamentos de Reservoir Computing: da intuição ao modelo mínimo

Resumo. *Este artigo apresenta Reservoir Computing (RC) como o primeiro grande degrau teórico da série. O foco não é apenas definir o termo, mas mostrar como um reservatório transforma uma sequência de entradas em uma representação dinâmica rica e por que isso e útil em previsão temporal. Partimos da intuição de memória distribuída, formalizamos o modelo mínimo com equações, conectamos cada elemento do formalismo a arquivos reais do projeto e mostramos artefatos computacionais intermediários do caso Favorita. Ao final, o leitor consegue ler o código do RC clássico do projeto, entende o papel do* `washout`*, sabe o que é treinado e o que não é treinado e fica preparado para executar o primeiro pipeline completo no artigo seguinte.*

0.1. 1. O que o leitor vai aprender

Ao final deste artigo, você será capaz de:

1. definir um reservatório clássico de forma matemática;
2. distinguir estado interno, vetor de entrada e readout;
3. explicar o papel de `spectral_radius`, `leak_rate` e `washout`;
4. mapear as equações de RC para a implementação em `code/rc/model.py`;
5. interpretar artefatos intermediários, e não apenas métricas finais.

0.2. 2. A intuição: um reservatório e uma memória dinâmica

Em um modelo tabular tradicional, as features costumam ser calculadas explicitamente: lags, médias moveis, indicadores de calendário e assim por diante. Em RC, a ideia e diferente. Em vez de escrever manualmente toda a representação temporal, injetamos a sequência em um sistema dinamico recorrente e usamos seus estados internos como uma base de representação.

A intuição mais útil para iniciantes e esta:

- a entrada u_t perturba o sistema;
- o estado x_t carrega um resumo dinamico do passado recente;
- o readout linear aprende a converter esse resumo em previsão.

RC é atraente didaticamente porque separa o problema em duas partes:

1. um mecanismo fixo de representação temporal;
2. uma camada simples de regressão treinável.

0.3. 3. Construindo o modelo mínimo de RC

Em muitos textos de RC, a equação do reservatório aparece pronta, em uma única linha. Para quem está vendo o tema pela primeira vez, isso costuma ser brusco demais. Aqui vamos fazer o caminho inverso: primeiro identificamos os ingredientes da atualização, depois construímos o estado candidato, e só ao final escrevemos a equação completa.

0.3.1. 3.1 Os ingredientes da atualização

Em cada instante t, o reservatório recebe uma entrada u_t e mantém internamente um estado x_{t-1} vindo do passo anterior. O objetivo da atualização é responder à pergunta:

> dado o que entrou agora e dado o que o sistema ainda "lembra"do passado,
> qual deve ser o novo estado interno?

Para responder a essa pergunta, o modelo usa:

- $u_t \in \mathbb{R}^d$, o vetor de entrada no tempo t;
- $x_{t-1} \in \mathbb{R}^N$, o estado interno do passo anterior;
- W_{in}, a matriz que projeta a entrada para o espaço do reservatório;
- W_{res}, a matriz recorrente interna, que mistura os componentes do próprio estado;
- b, um vetor de bias;
- $\alpha \in (0, 1]$, o `leak_rate`, que controlará quanto do estado novo realmente entra.

Antes de pensar em "novo estado", vale notar que o reservatório olha para duas fontes de informação ao mesmo tempo:

1. a entrada atual, que chega por $W_{in}u_t$;
2. a memória interna, que chega por $W_{res}x_{t-1}$.

0.3.2. 3.2 Primeiro passo: combinar passado e entrada

O primeiro passo é formar uma ativação bruta, que ainda não é o novo estado final. Chamaremos essa quantidade de pré-ativação:

$$z_t = W_{res}x_{t-1} + W_{in}u_t + b.$$

Essa expressão deve ser lida com calma:

- $W_{res}x_{t-1}$ leva a informação do passado para o presente;
- $W_{in}u_t$ injeta a entrada atual no reservatório;
- b desloca a resposta de cada unidade.

O vetor z_t representa, portanto, a resposta linear bruta do sistema antes de qualquer limitação ou filtragem.

0.3.3. 3.3 Segundo passo: obter o estado candidato

Se usássemos diretamente z_t como novo estado, o modelo permaneceria linear demais e os valores internos poderiam crescer sem controle. Por isso, aplicamos uma não linearidade componente a componente:

$$\widetilde{x}_t = \tanh(z_t).$$

Substituindo z_t, obtemos:

$$\widetilde{x}_t = \tanh(W_{res}x_{t-1} + W_{in}u_t + b).$$

O vetor $\widetilde{x}_t$ é chamado aqui de **estado candidato**. Ele representa a resposta que o reservatório produziria se reagisse imediatamente ao passo atual. A função $\tanh(\cdot)$ comprime os valores para o intervalo $(-1, 1)$, introduz não linearidade e ajuda a manter a dinâmica interna em uma faixa estável.

Em linguagem intuitiva:

- z_t é a ativação bruta;
- $\widetilde{x}_t$ é a proposta de novo estado depois da filtragem não linear.

0.3.4. 3.4 Terceiro passo: misturar memória e atualização nova

O reservatório ainda não adota $\widetilde{x}_t$ como estado final. Em vez disso, ele mistura parte da memória anterior com parte do estado candidato:

$$x_t = (1 - \alpha)x_{t-1} + \alpha\widetilde{x}_t.$$

Essa é uma equação importante pedagogicamente. Ela mostra que o reservatório não "apaga"o passado a cada passo. O parâmetro α controla a velocidade da atualização:

- se α é pequeno, o sistema preserva mais memória e muda devagar;
- se α é grande, o sistema reage mais ao presente e muda mais rápido.

0.3.5. 3.5 A equação completa do reservatório

Agora sim podemos juntar os passos anteriores. Substituindo o estado candidato na equação de mistura, chegamos à dinâmica completa:

$$x_t = (1 - \alpha)x_{t-1} + \alpha \tanh(W_{res}x_{t-1} + W_{in}u_t + b).$$

Nessa forma final:

- u_t é a entrada atual;
- x_{t-1} é a memória interna trazida do passo anterior;
- W_{res} modela a recorrência interna do reservatório;
- W_{in} projeta a entrada para o espaço de estados;
- b desloca a ativação;
- α controla o equilíbrio entre memória e atualização.

No código, essa construção aparece de forma quase literal em `EchoStateReservoir.step()`:

```
pre_activation = (
    self.recurrent_weights @ self.state
    + self.input_weights @ input_vector
```

```
    + self.bias
)
new_state = np.tanh(pre_activation)
self.state = (1.0 - self.config.leak_rate) * self.state + self.config.l
```

A correspondência entre teoria e implementação fica mais fácil de ler quando usamos os mesmos três passos:

1. `pre_activation` corresponde a z_t;
2. `new_state` corresponde a $\widetilde{x}_t$;
3. `self.state = ...` corresponde à atualização final de x_t.

0.3.6. 3.6 Como a previsão sai do estado interno

A dinâmica do reservatório produz uma representação temporal, mas ainda não produz a previsão final. Para isso, usamos um readout linear:

$$\hat{y}_t = W_{out}\phi_t, \qquad \phi_t = [1; u_t; x_t].$$

O vetor ϕ_t concatena três elementos:

- um termo constante, representado por `1`;
- a entrada atual u_t;
- o estado interno x_t.

No projeto, `\phi_t` é construída por `_feature_vector()` e `W_out` é ajustado via regressão Ridge.

0.4. 4. O que é treinado e o que não é treinado

Um dos pontos mais didáticos de RC e a divisão entre dinâmica fixa e leitura treinável.

No projeto:

- `W_in`, `W_res` e `b` sao inicializados aleatoriamente e mantidos fixos;
- apenas o readout linear e ajustado com supervisão.

Formalmente, o treinamento resolve

$$W_{out}^* = \arg\min_{W_{out}} \|Y - \Phi W_{out}\|_2^2 + \lambda \|W_{out}\|_2^2,$$

em que Φ é a matriz de features acumuladas ao longo do tempo e λ é o hiperparametro de regularização Ridge.

Essa escolha reduz o custo de treinamento em relação a RNNs totalmente treináveis e, ao mesmo tempo, preserva uma representação temporal rica.

0.5. 5. Como o tempo entra no modelo

No projeto, o vetor de entrada sequencial e construído em `code/common/sequential_features.py` com sete componentes:

- demanda prévia normalizada;
- promoção normalizada;
- `dow_sin`, `dow_cos`;
- `month_sin`, `month_cos`;
- indicador de fim de semana.

Em notação compacta,

$$u_t = \begin{bmatrix} \widetilde{y}_{t-1} & \widetilde{p}_t & \sin(2\pi\,\mathrm{dow}_t/7) & \cos(2\pi\,\mathrm{dow}_t/7) & \sin(2\pi\,\mathrm{month}_t/12) & \cos(2\pi\,\mathrm{month}_t/12) & \mathbb{1}_{\mathrm{weekend}} \end{bmatrix}^\top$$

Esse vetor coloca em uma mesma representação:

- memória de curtissimo prazo, via $\widetilde{y}_{t-1}$;
- contexto exógeno, via promoção;
- ciclo temporal, via senos e cossenos.

0.6. 6. O papel do washout

Nos primeiros instantes da simulação, o estado do reservatório ainda esta fortemente contaminado pela inicialização. Por isso, o projeto descarta os primeiros passos antes de treinar o readout.

Se chamarmos o `washout` de w, o treino do readout usa apenas os tempos $t \geq w$:

$$\Phi = [\phi_w, \phi_{w+1}, \ldots, \phi_T]^\top.$$

Em `fit_rc_readout()`, isso aparece no teste `if idx >= config.washout`.

O `washout` não é um detalhe de implementação. Ele e uma parte conceitual do pipeline, porque separa a transiente inicial da dinâmica que realmente queremos aproveitar.

0.7. 7. Como o projeto implementa RC no caso Favorita

A tabela abaixo resume o pipeline implementado.

Objeto	Dimensão no projeto	Origem
Vetor de entrada u_t	7	`scaled_input_vector()`
Estado do reservatório x_t	80	`RCConfig.n_reservoir`
Vetor de features $[1; u_t; x_t]$	88	`_feature_vector()`
Readout	1 saida	Ridge linear sobre as features

O fluxo completo do código e o seguinte:

1. `build_scaler_state()` calcula médias e desvios para normalizacao;
2. `scaled_input_vector()` constroi u_t;
3. `EchoStateReservoir.step()` atualiza x_t;
4. `_feature_vector()` monta $\phi_t = [1; u_t; x_t]$;
5. `Ridge.fit()` estima o readout;
6. `forecast_rc()` faz previsão recursiva no bloco de teste.

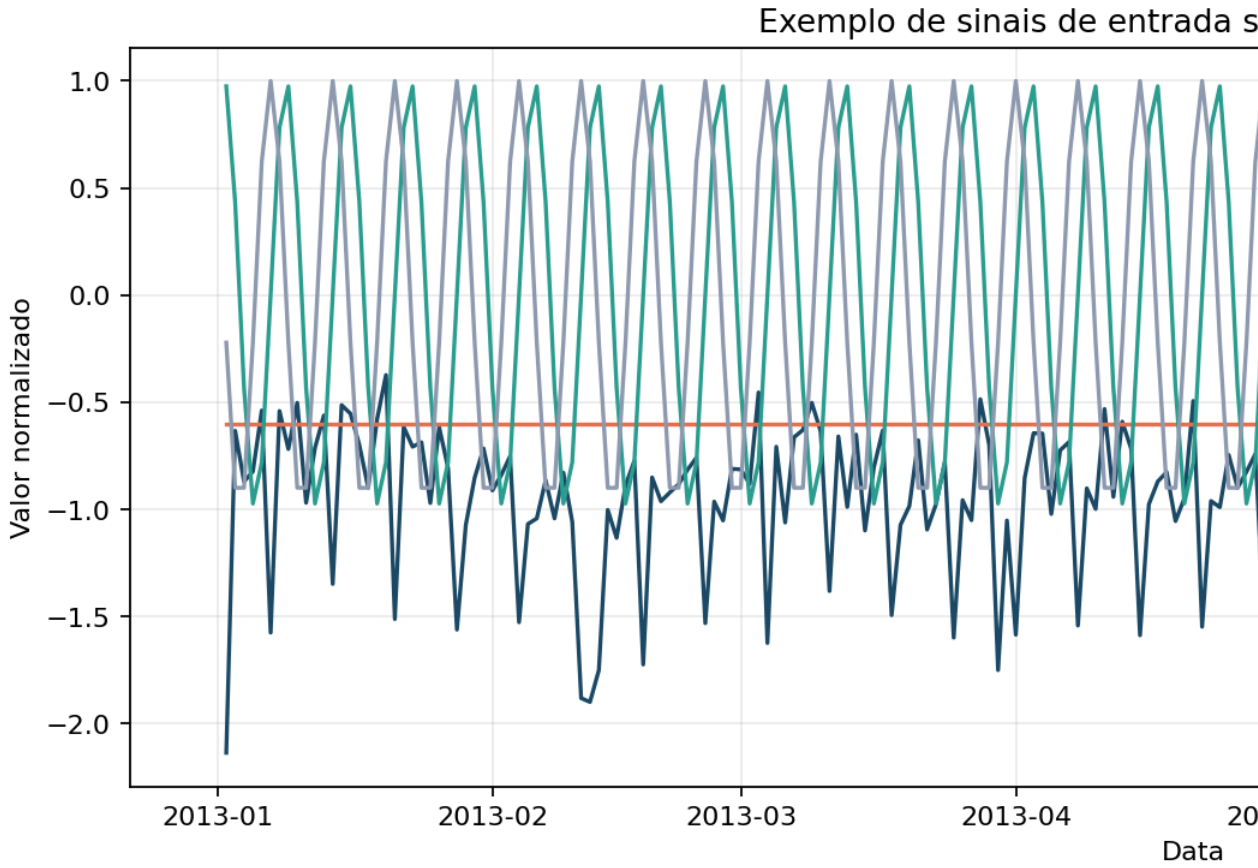
O trecho central do treinamento e este:

```
input_vector = scaled_input_vector(prev_y, row, scaler=scaler)
state = reservoir.step(input_vector)
feature_rows.append(_feature_vector(state, input_vector))
targets.append(float(row["y"]))
```

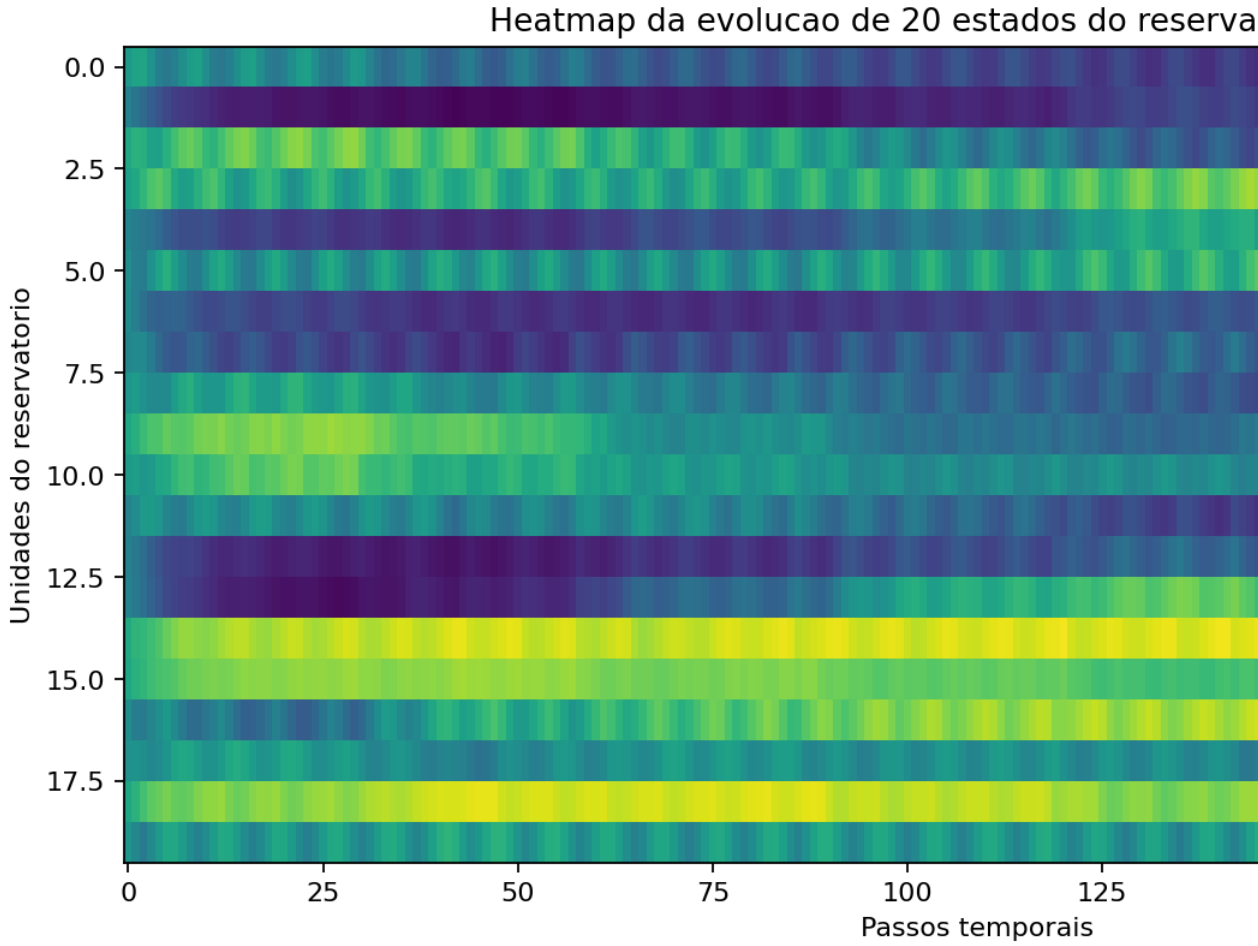
Isso mostra a ideia central de RC em sua forma mais simples: o modelo não aprende uma representação temporal diretamente por backpropagation; ele coleta estados dinamicos e aprende apenas como le-los.

0.8. 8. O que os artefatos intermediários mostram

O artigo não precisa esperar a métrica final para ensinar alguma coisa. Os artefatos computacionais deste modulo mostram o pipeline em um nivel mais interno.



A imagem de entrada sequencial mostra que o reservatório não recebe apenas uma série de vendas crua. Ele recebe uma codificacao temporal e exógena compacta.



O heatmap dos estados ilustra duas ideias importantes:

1. unidades diferentes respondem de modos diferentes ao mesmo estimulo;
2. a memória do reservatório não é um unico lag explícito, mas uma combinacao distribuída de estados.

Em outras palavras, RC constroi uma base dinâmica de features sem precisar enumerar manualmente toda a interação temporal.

0.9. 9. Comparando RC com outras famílias de modelo

Vale situar RC entre outros paradigmas que também aparecem na série:

- `Seasonal Naive`, `ETS` e `Prophet` modelam estrutura temporal de forma mais explícita;
- `XGBoost` usa engenharia de atributos tabulares;
- `LSTM` aprende dinâmica recorrente via gradiente;
- `RC` fixa a dinâmica recorrente e treina apenas o readout.

Essa posicao intermediária e pedagogicamente muito poderosa. RC é mais rico do que um baseline ingenuo, mas muito mais simples de treinar do que uma RNN completa.

0.10. 10. Passo-a-passo para estudar ou modificar o RC do projeto

Um leitor que queira estudar a implementação deve seguir esta ordem:

1. abrir `code/common/sequential_features.py`;
2. verificar como o vetor `u_t` e construído;
3. abrir `code/rc/model.py` e localizar `RCConfig`;
4. ler `EchoStateReservoir.step()` junto da equação do artigo;
5. ler `fit_rc_readout()` e localizar o `washout`;
6. executar `python code/rc/run.py`;
7. validar o comportamento com `pytest code/rc/test_model.py`.

Essa ordem importa. Se o leitor abrir apenas `run.py`, perde a correspondencia entre teoria e implementação.

0.11. 11. Conclusão

RC já esta completamente formulado no contexto do projeto. Temos uma definicao matemática, uma implementação concreta e artefatos intermediários que mostram a representação temporal em acao. O próximo passo natural e colocar esse modelo em confronto com um baseline forte, medindo o que ele realmente entrega no recorte adotado no Favorita.

0.12. Entregaveis associados no repositorio

- implementação do RC: `code/rc/model.py`
- execucao do RC: `code/rc/run.py`
- testes do RC: `code/rc/test_model.py`
- construcao de entrada sequencial: `code/common/sequential_features.py`
- artefatos deste artigo: `computational_results_20260402_222902/`

0.13. Referencias

- Jaeger, H. The "echo state"approach to analysing and training recurrent neural networks.
- Lukosevicius, M.; Jaeger, H. Reservoir computing approaches to recurrent neural network training.
- Lukosevicius, M. A practical guide to applying echo state networks.
- Fujii, K.; Nakajima, K. Harnessing disordered-ensemble quantum dynamics for machine learning.